



计算问题的复杂性

——Computability Theory 华中农大 信息学院

华中农业大学信息学院

本章内容

- 将介绍还没有找到可以用有效算法加以解决的一类计算问题
 - P类
 - NP类
 - NP完全问题
 - co-NP类
 - NPI类

10.1 引言

- 设 Π 是任意问题，如果对问题 Π 存在一个算法，它的时间复杂性是 $O(n^k)$ ，其中 n 是输入大小， k 是非负整数，我们说存在着求解问题 Π 的**多项式时间算法**。这类算法在**可以接受的时间**内实现问题求解， e.g., 排序、串匹配、矩阵相乘.
- 但是，现实世界中的许多有趣问题并不属于这个范畴，因为求解这些问题所需要的时间量要用**指数和超指数函数**(如 2^n 和 $n!$) 来测度。随着问题规模的增长而**快速增长**。
- 在计算机科学界已达成这样的共识，认为存在多项式时间算法的问题是**易求解**的，而对于那些不大可能存在多项式时间算法的问题是**难解**的。

易解问题与难解问题的主要区别

在学术界已达成这样的共识：把多项式时间复杂性作为易解问题与难解问题的分界线，主要原因有：

1) 多项式函数与指数函数的增长率有本质差别

问题规模 n	多项式函数					指数函数	
	logn	n	nlogn	n²	n³	2ⁿ	n!
1	0	1	0.0	1	1	2	1
10	3.3	10	33.2	100	1000	1024	3628800
20	4.3	20	86.4	400	8000	1048376	2.4E18
50	5.6	50	282.2	2500	125000	1.0E15	3.0E64
100	6.6	100	664.4	10000	1000000	1.3E30	9.3E157

2) 计算机性能的提高对易解问题与难解问题算法的影响

假设求解同一个问题有5个算法A1~A5，时间复杂度 $T(n)$ 如下表，假定计算机C2的速度是计算机C1的10倍。下表给出了在相同时间内不同算法能处理的问题规模情况：

	$T(n)$	n	n'	变化	n'/n
A1	$10n$	1000	10000	$n'=10n$	10
A2	$20n$	500	5000	$n'=10n$	10
A3	$5n\log n$	250	1842	$n < n' < 10n$	7.37
A4	$2n^2$	70	223	n	3.16
A5	2^n	13	16	$n'=n+\log 10 \approx n+3$	≈ 1

3) 多项式时间复杂性忽略了系数，不影响易解问题与难解问题的划分

问题规模n	多项式函数			指数函数	
	n^8	$10^8 n$	n^{1000}	1.1^n	$2^{0.01n}$
5	390625	5×10^8	5^{1000}	1.611	1.035
10	10^8	10^9	10^{1000}	2.594	1.072
100	10^{16}	10^{10}	10^{2000}	13780.6	2
1000	10^{24}	10^{11}	10^{3000}	2.47×10^{41}	1024

观察结论： $n \leq 100$ 时，（不自然的）多项式函数值大于指数函数值，
但n充分大时，指数函数仍然超过多项式函数。

难解问题

- 这一类含有许许多多问题，其中还包含了数百个著名的问题，它们有一个共同的特性，即如果它们中的一个是多项式可解的，那么所有其他的问题也是多项式可解的。
- 此外，现存的求解这些问题的算法的运行时间，对于中等大小的输入也要用几百或几千年来测度。

判定问题和最优化问题

- 判定问题:
 - 在研究NP完全性理论时，我们很容易重述一个问题使它的解只有两个结论:yes或no，在这种情况下，称问题为判定问题。
- 最优化问题:
 - 与此相对照，最优化问题是关心某个量的最大化或最小化的问题。在前面的章节中，已经遇到过大量的最优化问题，像找出一张表中的最大或最小元素的问题，在有向图中寻找最短路径问题和计算一个无向图的最小生成树的问题。
- 如果我们有一个求解判定问题的有效算法，那么很容易把它变成求解与它相对应的最优化问题的算法。反之亦然。

例子 10.1



设 S 是一个实数序列, ELEMENT UNIQUENESS问题为, 是否 S 中的所有数都是不同的。

- **判定问题:** ELEMENT UNIQUENESS.

输入: 一个整数序列 S

问题: 在 S 中存在两个相等的元素吗?

- **最优问题:** ELEMENT COUNT

输入: 一个整数序列 S

输出: 一个在 S 中频度最高的元素。

这个问题可以用显而易见的方法在最优的时间 $O(n \log n)$ 解决, 这意味着它是易解的。

例子 10.2



给出一个无向图 $G = (V, E)$ ，用 k 种颜色对 G 着色是这样的问题：对于 V 中的每一个顶点用 k 种颜色中的一种对它着色，使图中没有两个邻接顶点有相同的颜色。

这个问题是难解的。

例子 10.2



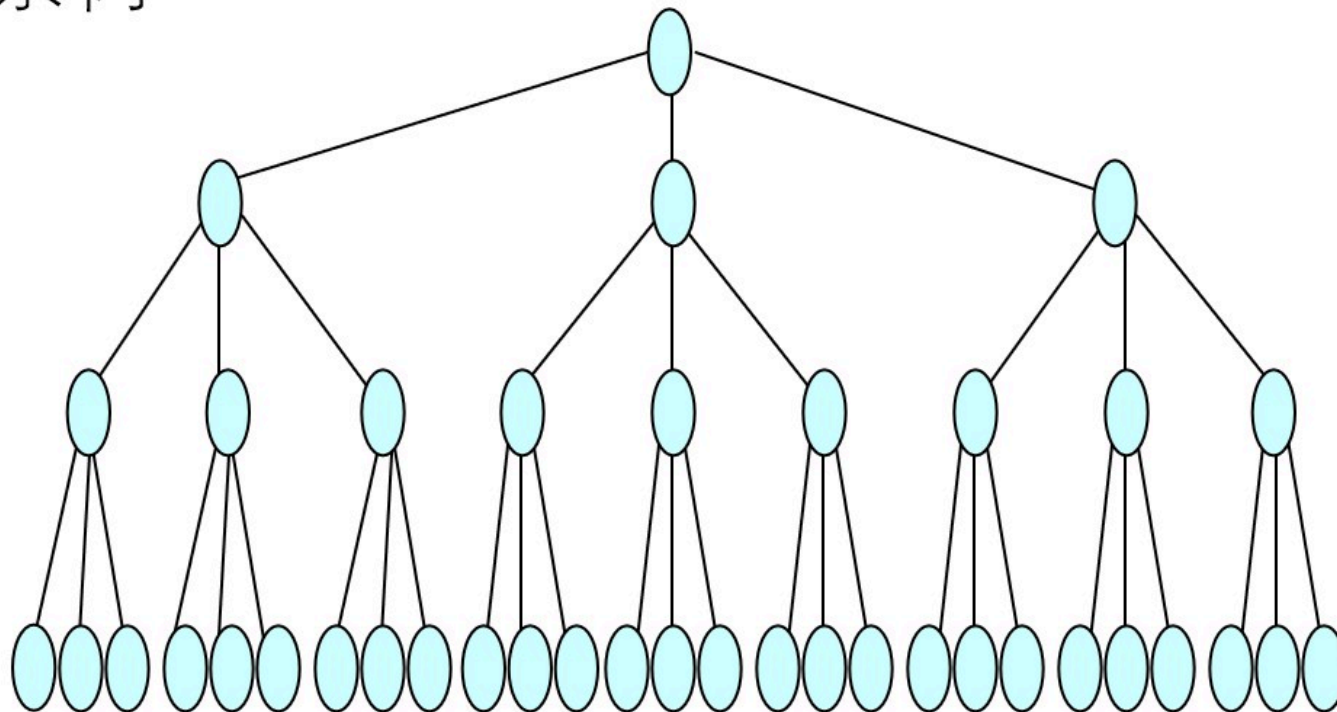
问题描述

- 如果 k 限于3, 问题就是非常著名的3着色问题。
- 给出一个无向图 $G=(V,E)$, 需要用三种颜色之一为 v 中的每个顶点着色, 三种颜色分别为1,2和3, 使得没有两个邻接的顶点有同样的颜色。
- 一种着色可以用 n 元组 $(c_1, c_2, \dots, c_n)$ 来表示, 使 $c_i \in \{1, 2, 3\}$, $1 \leq i \leq n$.
 - 例如, $(1,2,2,3,1)$ 表示一个有5个顶点的图的着色, 一个 n 个顶点的图共有 3^n 种可能的着色(合法的和非法的)。

搜索树

- 所有可能的着色的集合可以用一棵完全的三叉树来表示，称为**搜索树**。
- 在这棵树中，从根到叶节点的每一条路径代表一种**着色指派**。

搜索树

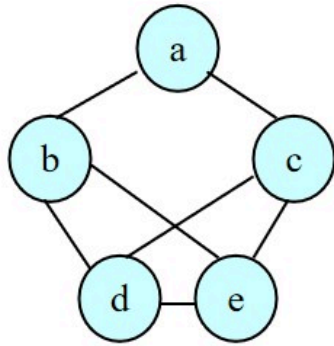


Backtracking: Partial Coloring Scheme

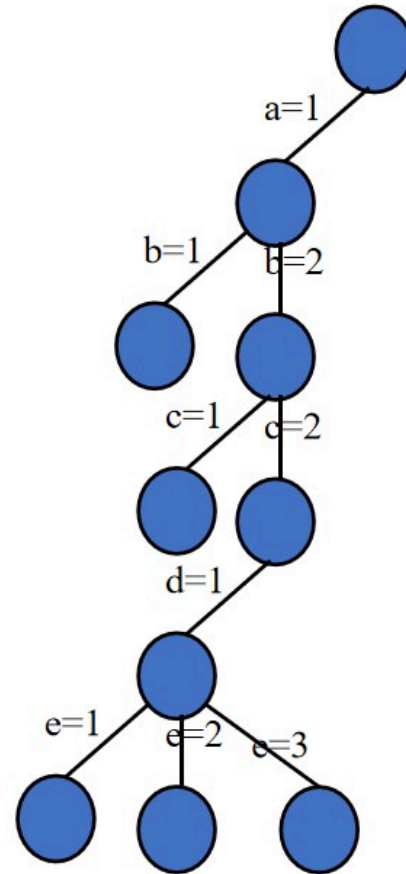
- 如果没有两个邻接的着色顶点有同样的颜色，图的一个不完全着色称为**部分解**。
- **Idea:**回溯法通过每次一个节点地生成基底树来工作。
 - 如果从根到当前节点的路径对应于一个合法着色过程就终止(除非期望找到不止一种的着色)
 - 如果这条路径的长度小于 n ，并且相应的着色是部分的，那么就生成现节点的一个子节点，并将它标记为**现节点**。
 - 另一方面，如果对应的路径不是部分的，那么现节点标识为**死节点**并生成对应于另一种颜色的**新节点**。
 - 如果所有三种颜色都已经试过且没有成功，搜索就回溯到父节点，它的颜色被改变，依次类推。



An example

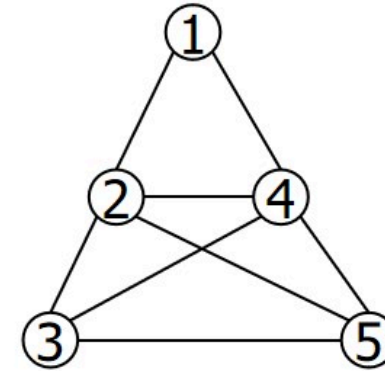


Find a solution!



例子 10.2 COLORING问题（图着色问题）

- 输入: (着色数) k , (节点数)5, (图的边) $(1,2)(1,4)...$
- 写出长度为 n (节点数)的字符串,例如,RGRBG, RGRBO, RBYGO, RGRBY, R%*\$@等, 至少有 k^n 个不同着色情况。
- 找到符合要求（任何两个邻接顶点颜色不同）的最小的 k 值



例子 10.2 (Continue)



- 判定问题: COLORING

输入: 一个无向图 $G = (V, E)$ 和一个正整数 $k \geq 1$ 。

问题: G 可以 k 着色吗? 即 G 可以用 k 种颜色着色吗?

- 这个问题的一个最优形式是, 对一个图着色, 使图中没有两个邻接的顶点有相同的颜色, 所需要的最少颜色数是多少? 这个数记为 $\chi(G)$, 称为 G 的色数。

- 最优化问题: CHROMATIC NUMBER

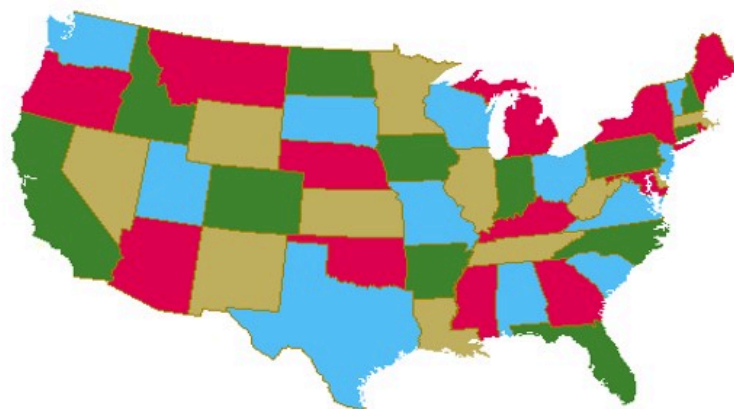
输入: 一个无向图 $G = (V, E)$ 。

输出: G 的色数。

四色猜想

- 地图四色定理(Four color theorem)最先是由一位叫古德里 (Francis Guthrie) 的英国大学生来的。

四色问题的内容是：“任何一张地图只用四种颜色就能使具有共同边界的国家着上不同的颜色。”



四色猜想

- 1852年，刚从伦敦大学毕业的Francis Guthrie提出了四色猜想。
- 1878年著名的英国数学家Cayley向数学界征求解答。
- 此后数学家 Heawood 花费了毕生的精力致力于四色研究，于1890年证明了五色定理（每个平面图都是5顶点可着色的）。
- 直到1976年6月，美国数学家 K. Appel与 W. Haken，在3台不同的电子计算机上，用了1200小时，作了100亿判断，才终于完成了“四色猜想”的计算机证明。

例子 10.3



给出一个无向图 $G=(V,E)$, 对于某个正整数 k , G 中大小为 k 的团集, 是指 G 中有 k 个顶点的一个完全子图。团集问题是问一个无向图是否包含一个预定大小的团集。

- **判定问题:** CLIQUE.

输入: 一个无向图 $G=(V,E)$ 和一个正整数 k 。

问题: G 有大小为 k 的团集吗?

- **最优化问题:** MAX-CLIQUE.

输入: 一个无向图 $G=(V,E)$.

输出: 一个正整数 k , 它是 G 中最大团集的大小

判定问题与最优化问题的逻辑联系

- 如果我们有一个求解判定问题的有效算法，那么很容易把它变成求解与它相对应的最优化问题的算法。
- 例如，我们有一个求解图着色判定问题的算法A，则可以用二分搜索并且把算法A作为子程序来找出图G的色数。很清楚， $1 \leq \chi(G) \leq n$ ，这里n是G中顶点数，因此仅用 $O(\log n)$ 次调用算法A就可以找到G的色数。由于我们正处理多项式时间的算法， $\log n$ 因子是不重要的。
- 因为这个理由，在NP完全问题的研究中，甚至在一般意义上的计算复杂性或可计算性的研究中，把注意力限制在判定问题上会更容易一些

10.2 P类

因此，易解问题和难解问题的划分标准可基于对所谓判定问题的求解方式。

事实上，实际应用中的大部分问题可以很容易转化为相应的判定问题，如：

- 排序问题 $\Rightarrow$ 给定一个实数数组，是否可以按非降序排列？
- 图着色问题：给定无向连通图 $G=(V,E)$ ，求最小色数 k ，使得任意相邻顶点具有不同的着色 $\Rightarrow$

给定无向连通图 $G=(V,E)$ 和正整数 k ，是否可以用 k 种颜色.....？

10.2 P类

• 定义 10.1

- 设A是求解问题 Π 的一个算法，如果在展示问题 Π 的一个实例时，在整个执行过程中，每一步都只有一种选择，则称A是**确定性算法**。因此如果对于同样的输入，实例一遍又一遍地执行，它的输出从不改变。
- 特点：对同一输入实例，运行算法A，所得结果是一样的。

10.2 P类



• 定义 10.2

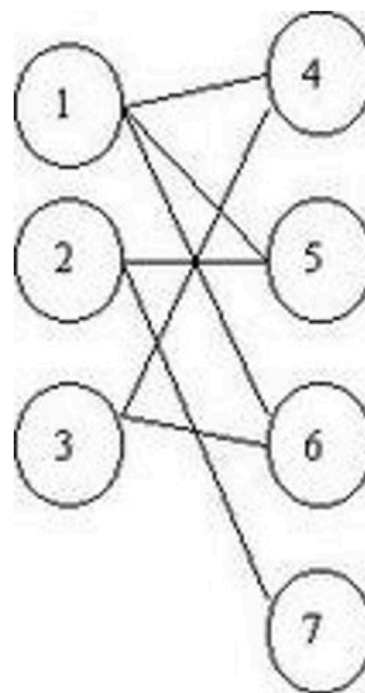
- **判定问题的P类**由这样的判定问题组成，它们的yes/no解可以用确定性算法在运行多项式步数内，例如在 $O(n^k)$ 步内得到，其中 k 是某个非负整数， n 是输入大小。
- 也就是说：如果对于某个判定问题 Π ，存在一个非负整数 k ，对于输入规模为 n 的实例，能够以 $O(n^k)$ 的时间运行一个确定性算法，得到yes或no的答案，则称该判定问题 Π 是一个P(Polynomial)类问题。
- 事实上，所有易解问题都是P类问题。

P类的问题

- 排序问题:
 - 给出一个 n 个整数的表, 它们是否按非降序排列?
- 不相交集问题:
 - 给出两个整数集合, 它们的交集是否为空?
- 最短路径问题:
 - 给出一个边上有正权的有向图 $G = (V, E)$, 两个特异的顶点 $s, t \in V$ 和一个正整数 k , 在 s 到 t 间是否存在一条路径, 它的长度最多是 k 。

P类的问题

- **2着色问题:**
 - 给出一个无向图 G ，它是否是2可着色的?即它的顶点是否可仅用两种颜色着色，使两个邻接顶点不会分配相同的颜色?注意，当且仅当 G 是二分图，即当且仅当它不包含奇数长的回路时，它是2可着色的。
- **2可满足问题:**
 - 给出一个合取范式(CNF)形式的布尔表达式 f ，这里每个子句恰好由两个文字组成，问 f 是可满足的吗?





- 如果对于任意问题 C , Π 的补也在 C 中, 我们说问题类 $\Pi \in C$ 在补运算下是封闭的。
 - 例如, 2着色问题的补可以陈述如下: 给出一个图 G , 它是不2可着色的吗? 我们称这个问题为NOT-2-COLOR问题。它是属于 P 的。
- 定理 10.1
 - P 类问题在补运算下是封闭的。

10.3 NP类

- **NP类**由这样的问题 Π 组成，对于这些问题存在一个确定性算法A，该算法在对 Π 的一个实例展示一个断言解时，它能在**多项式时间内验证解的正确性**。即如果断言解导致答案是yes，就存在一种方法可以在多项式时间内**验证这个解**。

不确定性算法

- 对于输入 x ，一个不确定性算法由下列两个阶段组成：
 - 猜测阶段
 - 验证阶段
- p177

猜测阶段

- 在这个阶段产生一个任意字符串 y ，它可能对应于输入实例的一个解，也可以不对应解。
- 事实上，它甚至可能不是所求解的合适形式，它可能在不确定性算法的不同次运行中不同。我们仅仅要求在多项式步数内产生这个串，即在 $O(n^i)$ 时间内，这里 $n=|x|$ ， i 是非负整数。对于许多问题，这一阶段可以在线性时间内完成。

验证阶段

- 在这个阶段，一个不确定性算法验证两件事
 - 首先，它检查产生的解串 y 是否有合适的形式，
 - 如果不是，则算法停下并回答no;
 - 另一方面，如果 y 是合适形式，那么算法继续检查它是否是问题实例 x 的解，如果它确实是实例 x 的解，那么它停下并且回答yes，否则它停下并回答no。
- 我们也要求这个阶段在多项式步数内完成，即在 $O(n^j)$ 时间内，这里 j 是一个非负整数。

验证阶段

- 设A是问题 Π 的一个不确定性算法，我们说A接受问题 Π 的实例I，当且仅当对于输入I存在一个导致yes回答的猜测。换句话说，A接受I当且仅当可能在算法的某次执行上它的验证阶段将回答yes。
- 要强调的是，如果算法回答no，那么这并不意味着A不接受它的输入，因为算法可能猜测了一个不正确解。

不确定性算法与NP类问题

- 定义(不确定性算法): 设A是求解问题 Π 的一个算法, 如果算法A以如下**猜测+验证的方式**工作, 称算法A为不确定性(nondeterminism)算法:
- **猜测阶段**: 对问题的输入实例产生一个任意字符串y, 在算法的每次运行, y可能不同, 因此猜测是以不确定的形式工作。这个工作一般可以在线性时间内完成。
- **验证阶段**: 在这个阶段, 用一个确定性算法验证两件事: 首先验证猜测的y是否是合适的形式, 若不是, 则算法停下并回答no; 若是合适形式, 则继续检查它是否是问题x的解, 如果确实是x的解, 则停下并回答yes, 否则停下并回答no。要求验证阶段在多项式时间内完成。

不确定性算法与NP类问题

对不确定性算法输出yes/no的理解:

- 若输出no, 并不意味着不存在一个满足要求的解, 因为猜测可能不正确;
- 若输出yes, 则意味着对于该判定问题的某一输入实例, 至少存在一个满足要求的解。

NP类问题的形式定义

- 定义(**NP类问题**): 如果对于判定问题 Π , 存在一个非负整数 k , 对于输入规模为 n 的实例, 能够以 $O(n^k)$ 的时间运行一个**不确定性算法**, 得到**yes/no**的答案, 则该判断问题 Π 是一个NP(**N**ondeterministic **P**olynomial)类问题。

※注意: NP类问题是对于判定问题定义的, 事实上, 可以在多项式时间内应用非确定性算法解决的所有问题都属于NP类问题。

NP类问题的形式定义

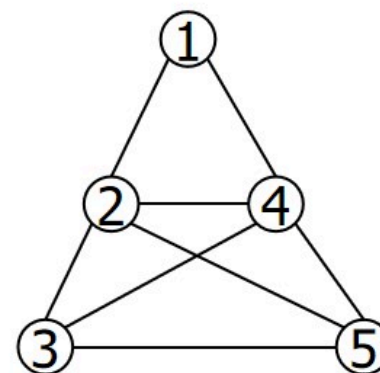


• 定义 10.3

- 判定问题类NP由这样的判定问题组成:对于它们存在着多项式时间内运行的不确定性算法。
- 至于一个(不确定性)算法的运行时间,它仅仅是两个运行时间的和:一个是猜测阶段的时间,另一个是验证阶段的时间。因此它是 $O(n^i)+O(n^j)=O(n^k)$, k 是某个非负整数。

例子（图着色问题）

- Input: (着色数) k , (节点数)5, (图的边) (1,2)(1,4)...
- **猜测阶段** 指写出长度为 n (节点数)的字符串,例如, RGRBG, RGRB, RBYGO, RGRBY等等. 至少有 k^n 个猜测。
- **验证阶段** 指检查一个猜测的字符串是否合法（即是否是一个 k -着色）。
- 如果猜测字符串的时间是 $O(p_1(n))$, 验证字符串的时间是 $O(p_2(n))$; 而且 $p_1(n)$, $p_2(n)$ 是 n 的多项式(从而也是输入长度的多项式), 则该不确定算法有多项式时间。
- 如果输入的图能 k 着色则不确定算法停机并给出yes回答。



如何证明NP类问题——以着色问题为例



- 考虑COLORING问题，我们用两种方法证明这个问题属于NP类。
- 方法1:(非形式化的)
 - 设I是COLORING问题的一个实例，s被宣称为I的解。
 - 容易建立一个确定性算法来验证，是否确实是I的解。
 - 从NP类的非形式定义可得COLORING问题属于NP类。

方法 2

建立不确定性算法

- 当图G用编码表示后，一个算法A可以很容易地构建并运作如下：
 - 首先通过遍历顶点集合产生一个任意的颜色指派以“猜测”一个解。
 - 接着，A验证这个指派是否是有效的指派，如果它是一个有效的指派，那么A停下并且回答yes，否则它停下并回答no。
 - 请注意，根据不确定性算法的定义，仅当对问题的实例回答是yes时，A回答yes。
 - 其次是关于需要的运行时间，A在猜测和验证两个阶段总共花费不多于多项式时间。

关于P与NP关系的初步思考

- 1) 若问题 Π 属于P类，则存在一个多项式时间的确定性算法，对它进行判定或求解；显然，也可以构造一个多项式时间的非确定性算法，来验证解的正确性，因此，问题也属NP类。因此，显然有

$$P \subseteq NP$$

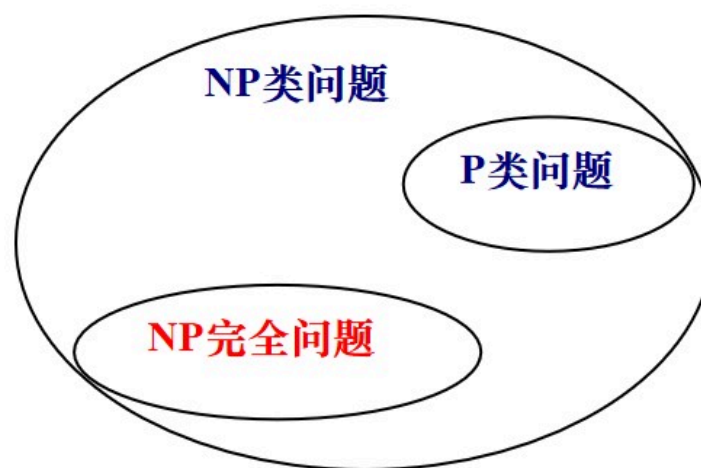
- 2) 若问题 Π 属于NP类，则存在一个多项式时间的非确定性算法，来猜测并验证它的解；但不一定能构造一个多项式时间的确定性算法，来对它进行求解。

- 因此，人们猜测 $P \neq NP$ ，但是否成立，至今未得到证明。
- $P=NP?$ 是计算机科学中最大的问题之一

10.4 NP完全问题

- 术语“NP完全”表示NP判定问题的一个子类，它们在下述意义上是最难的，即如果它们中的一个被证明用多项式时间确定性算法可解，那么NP中的所有问题用多项式时间确定性算法可解，即 $NP=P$ 。
- NP完全问题是**NP类问题的子类**，一个具有特殊性质与特殊意义的子类。

P类问题、NP类问题、NP完全问题的关系



对“NP完全问题”的评述

- NP完全问题是NP类问题中最难的一类问题，至今已经发现了几千个，但一个也没有找到多项式时间算法。
- 如果某一个NP完全问题能在多项式时间内解决，则每一个NP完全问题都能在多项式时间内解决。
- 这些问题也许存在多项式时间算法，因为计算机科学是相对新生的科学，肯定还有新的算法设计技术有待发现；
- 这些问题也许不存在多项式时间算法，但目前缺乏足够的依据来证明这一点。



10.4.1 可满足性问题(SAT问题)

- 给出布尔公式 f ，如果它是子句的合取，我们说它是合取范式(CNF)。一个子句是文字的析取，这里文字是一个布尔变元或它的否定。一个这种公式的例子是，参考教材178
- 一个公式称为是可满足的，如果对它的变元存在一个真值的指派使它为真。例如，上面的公式是可满足的，因为在 x_1 和 x_3 都置为真的任意指派下它为真。

判定问题： SATISFIABILITY

输入：一个合取范式的布尔公式 f 。

问题： f 是可满足的吗？

10.4.1 可满足性问题(SAT问题)

可满足性问题被证明是NP完全的第一个问题。作为第一个NP完全问题，还不存在可归约到其他的NP完全问题。因此要证明NP类中的所有问题都可以在多项式时间内归约到它。

定理 10.2 SATISFIABILITY问题是NP完全的。

(Cook定理, 1971) $\text{SAT} \in \text{NPC}$.

计算复杂性理论的奠基性工作之一：第一个被证明的NPC问题！是证明许多其他NPC问题的出发点

前面已经证明 $\text{SAT} \in \text{NP}$ ，所以尚需证明：
任何一个NP问题可以多项式归约为SAT

基本思想：

- ✓ 若 $A \in \text{NP}$ ，则A存在非多项式时间算法（猜测解、验证解）。
- ✓ 对猜测解、验证解的过程进行分析，构造一个SAT问题！

可规约关系的传递性

下面的定理说明可归约性关系 $\propto_p$ 是传递的，这在证明其他问题是NP完全时是必要的。我们说明这一点如下，假设对于NP中某个问题 Π ，我们可以证明在多项式时间内SATISFIABILITY可归约到 Π ，根据上面的定理，NP中的所有问题在多项式时间内可归约到SATISFIABILITY，因此，如果可归约关系 $\propto_p$ 是传递的，那么这就蕴含着NP中的所有问题在多项式时间内可归约到 Π 。

定理 10.3 设 Π ， Π' 和 Π'' 是三个判定问题，有 $\Pi \propto_{poly} \Pi'$ 和 $\Pi' \propto_{poly} \Pi''$ ，那么 $\Pi \propto_{poly} \Pi''$ 。

如何证明一个问题是NP完全的

推论 10.1 如果 Π 和 Π' 是NP中的两个问题，若有 $\Pi' \propto_{\text{poly}} \Pi$ ，并且 Π' 是NP完全的，则 Π 是NP完全的。 使用定理10.3证明

根据上面的推论，为了证明一个问题 Π 是NP完全的，仅需要证明下面两个条件同时成立。

(1) $\Pi \in \text{NP}$ 。

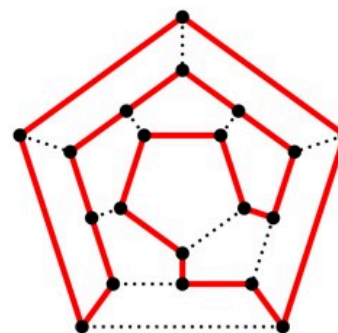
(2) 存在一个NP完全问题 Π' ，使 $\Pi' \propto_{\text{poly}} \Pi$

如何证明一个问题是NP完全的

例10.5

• 考虑下面两个问题

- **哈密顿回路问题**：给出一个无向图 $G=(V,E)$ ，它有哈密顿回路吗？即一条恰好访问每个顶点一次的回路
- **旅行商问题**：给出一个 n 个城市集合，且给出城市间距离。对于一个整数 k ，是否存在最长为 k 的游程？这里，一条游程是一个回路，它恰好访问每个城市一次



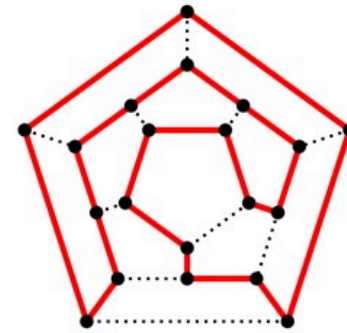
如何证明一个问题是NP完全的

众所周知，哈密顿问题是NP完全问题，我们将用这个事实来证明旅行商问题也是NP完全的

- 证明的第一步是说明旅行商问题是NP的，这非常简单，因为一个不确定性算法可以从猜测一个城市序列开始，然后验证这个序列是一个游程。如果是的，就继续看游程的长度是否超过给出的界 k

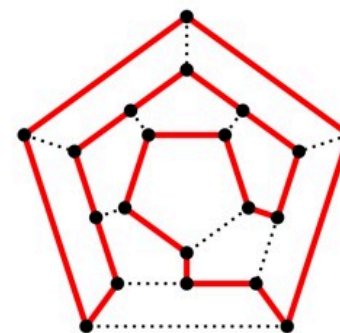
如何证明一个问题是NP完全的

- 第二步：证明哈密顿回路问题可以在多项式时间内规约到旅行商问题，即
 - 哈密顿回路问题 $\propto_{\text{poly}}$ 旅行商问题
- 设G是哈密顿回路的任意实例，我们构建一个含权图G' 和一个界限k，使得当且仅当G'有一个总长不超过k的游程时，G有一条哈密顿回路。



如何证明一个问题是NP完全的

- 设 $G=\langle V,E \rangle$, $G'=\langle V,E' \rangle$ 是顶点 V 集合上的完全图
- 接着, 我们给 E' 中的每条边指派一个权重如下
 - $l(e) = \begin{cases} 1 & \text{若 } e \in E \\ n & \text{若 } e \notin E \end{cases}$
- 其中 $n=|V|$ 。最后, 我们指派 $k=n$ 。从构建中很容易看到, 当且仅当 G' 有一个长度恰好为 n 的游程时, G 有一条哈密顿回路。



其它NP完全问题

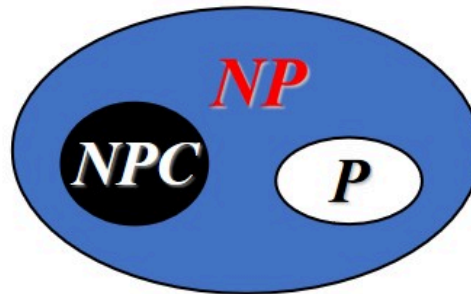
NPC问题的历史并不久，cook在1971年找到了第一个NPC问题，此后人们又陆续发现很多NPC问题，现在可能已经有3000多个了。所以，我们一般认为NPC问题是难解的问题，因为他不太可能存在一个多项式时间的算法（如果存在则所有的NP问题都存在多项式时间算法，这太不可思议了，但是也不是不可能）。

类似哈密顿回路/路径问题，旅行商问题，多处理机调度问题，背包问题，装箱问题，3着色问题等等很多问题都是NPC问题，所以都是难解的问题。

总结

- **P问题**：如果一个问题可以找到一个能在多项式的时间里解决它的算法，那么这个问题就属于P问题
- **NP问题**：NP问题是指可以在多项式的时间里验证一个解的问题，先猜出了一个解，然后在多项式运算复杂度步骤内可以验证这个解正确与否。
- **NP完全问题**：同时满足下面两个条件的问题就是NPC问题。首先，它得是一个NP问题；然后，所有的NP完全问题都可以约化到它。

P, NP, NPC关系

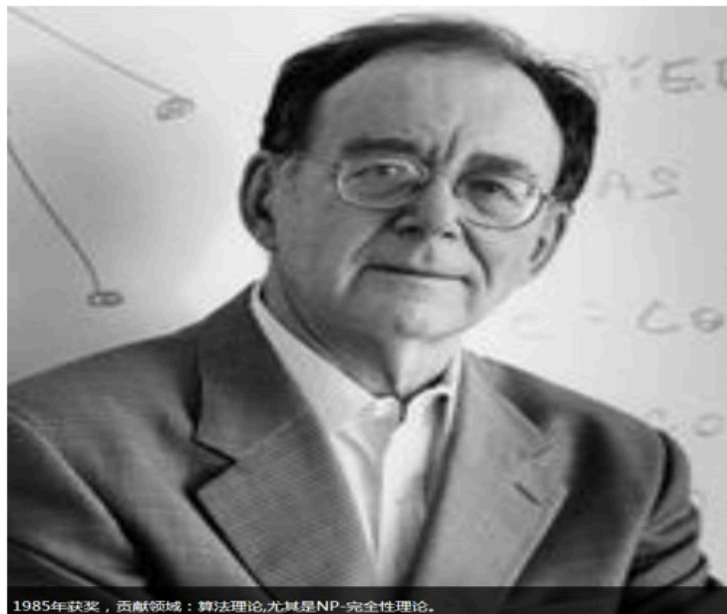


- (1) $P \subset NP$ 还是 $P = NP$? 是计算机科学中 最难的问题!!!
- (2) 任意一个NP-complete问题都属于NP-P?
 - 图同构问题?
 - 素数判定问题?

NP-Complete的证明



Karp的伟大贡献: 利用一个已知的NP-Complete问题, 通过规约, 证明另一个问题也是NP-Complete.



1985年获奖, 贡献领域: 算法理论, 尤其是NP-完全性理论。